

# 计算机系统基础实验

PA1

姓名: **伍知行** 学号: **251220081**

September 15, 2026

## 1 实验进度

我完成的实验内容如下:

1. 探究已有的源代码, 并尝试理解代码的整体框架;
2. 完善基础设施, 编写实现调试器的功能的代码;
3. 运行客户程序, 并使用所实现的调试功能来调试程序.

## 2 必答题

I. 尝试理解计算机如何计算:

从计算机的角度来看, 计算  $1 + 2 + \dots + 100$  的过程就是在不断重复“获取指令、解析指令、执行指令、访问内存、写回寄存器/内存”这个流程.

计算机只需严格遵循这样的流程, 就能一步一步地计算出我们所需要的结果.

II. 尝试画出计算  $1 + 2 + \dots + 100$  的状态机 (三元组表示  $(PC, r1, r2)$ ):

$(0, x, x) \rightarrow (1, 0, x)$

$\rightarrow (2, 0, 0) \rightarrow (3, 0, 1) \rightarrow (4, 1, 1)$ (第一次循环)

$\rightarrow (2, 1, 1) \rightarrow (3, 1, 2) \rightarrow (4, 3, 2)$ (第二次循环)

...

$\rightarrow (2, 4851, 98) \rightarrow (3, 4851, 99) \rightarrow (4, 4950, 99)$ (倒数第二次循环)

$\rightarrow (2, 4950, 99) \rightarrow (3, 4950, 100) \rightarrow (4, 5050, 100)$ (最后一次循环)

$\rightarrow (5, 5050, 100) \rightarrow (5, 5050, 100) \rightarrow \dots$

III. 实现 x86 的寄存器结构体:

我选择的指令系统是 riscv32, 因此目前无需完成此题.

IV. 理解 nemu 框架代码:

- nemu 主目录下有如下几个目录:

- 1) **build**: 此目录下的内容主要包括 nemu 构建好的系列程序以及相关运行日志;

具体而言, 当我们执行 `make run` 时, 就会运行当前子目录下的 `riscv32-nemu-interpret`(若是其他 ISA 就是对应 ISA 的 `interpreter`) 并编写相应日志;

```
wzhixing@ubuntu-22:~/ics2026/nemu$ make run
/home/wzhixing/ics2026/nemu/build/riscv32-nemu-interpret --log=/home/wzhixing/ics2026/nemu/build/nemu-log.txt
```

- 2) **include**: 此目录下主要包含着大量头文件, 如 `common.h`, `macro.h` 等, 在预处理时发挥作用;

- 3) **resource**: 一些辅助资源;

- 4) **src**: nemu 的核心代码部分, 此目录下包括着的核心子目录有:

– **device**: 此目录是关于设备配置的, 在 PA1 阶段中无需关注;

- **memory**: 此目录是关于内存设置的, 主要包括对物理地址 (*paddr*) 和虚拟地址 (*vaddr*) 的初始化、访问等;
- **cpu**: 此目录是关于 CPU 设置的, 包括 CPU 执行指令以及用于验证实现正确性的目录 *difftest*;
- **engine**: 此目录包括了解释器 (*interpreter*) 的实现;
- **isa**: 此目录下包括着各种指令集的相关配置、初始化;
- **monitor**: 此目录下是监视器模块的实现, 包括 *sdb* 和监视器主文件 *monitor.c*;
- **utils**: 此目录下包括着各类工具的实现, 如反汇编、日志、*nemu* 的状态定义、计时器等.

该目录下的其他核心文件还有 *nemu* 运行的主函数 *nemu-main.c* 以及相关 *make* 文件.

5) **scripts**: 这个目录下包括着 *Makefile* 构建的脚本文件;

6) **tools**: 这个目录下有很多辅助测试的工具, 在 PA1 阶段我们完成 *gen-expr.c* 并利用它生成测试表达式运算正确性的文件;

- *nemu* 主目录下的其他文件主要是关于配置的, 如 *Kconfig*、*.gitignore* 和 *Makefile* 等, 其他子目录下也有相应的配置文件, 在 *make* 时共同发挥作用.

## V. 优美地退出:

- 先观察输入 *q* 后发生的错误:

```
wzhixing@ubuntu-22:~/ics2026/nemu$ make run
/home/wzhixing/ics2026/nemu/build/riscv32-nemu-interpretor --log=/home/wzhixing/ics2026/nemu/build/nemu-log.txt
[src/utils/log.c:30 init_log] Log is written to /home/wzhixing/ics2026/nemu/build/nemu-log.txt
[src/memory/paddr.c:50 init_mem] physical memory area [0x80000000, 0x87ffffff]
[src/monitor/monitor.c:51 load_img] No image is given. Use the default build-in image.
[src/monitor/monitor.c:28 welcome] Trace: ON
[src/monitor/monitor.c:29 welcome] If trace is enabled, a log file will be generated to record the trace
. This may lead to a large log file. If it is not necessary, you can disable it in menuconfig
[src/monitor/monitor.c:32 welcome] Build time: 22:09:31, Sep  4 2026
Welcome to riscv32-NEMU!
For help, type "help"
(nemu) q
make: *** [/home/wzhixing/ics2026/nemu/scripts/native.mk:38: run] 错误 1
```

- 定位到相关文件 (包括 native.mk、state.c、nemu-main.c), 发现是 nemu 退出时返回值错误.

```
#include <utils.h>

NEMUState nemu_state = { .state = NEMU_STOP };

int is_exit_status_bad() {
    int good = (nemu_state.state == NEMU_END && nemu_state.halt_ret == 0) ||
               (nemu_state.state == NEMU_QUIT);
    return !good;
}
```

我们打印一下 nemu\_state, 发现它的值是 1, 即 NEMU\_STOP.

```
23 int main(int argc, char *argv[]) {
24     /* Initialize the monitor. */
25     #ifdef CONFIG_TARGET_AM
26     am_init_monitor();
27     #else
28     init_monitor(argc, argv);
29     #endif
30
31     /* Start engine. */
32     engine_start();
33
34     printf("NEMU_STATE: %d\n", nemu_state.state);
35     return is_exit_status_bad();
36 }
```

```
(nemu) q
NEMU_STATE: 1
make: *** [/home/wzhixing/ics2026/nemu/scripts/native.mk:38: run] 错误 1
```

这意味着我们需要在输入 q 时更改 nemu\_state 的值, 才能在

- 在 sdb.c 中找到 q 命令对应的函数 cmd\_q, 在该函数内部更改 nemu\_state 的值为 NEMU\_QUIT.

即将下图的函数

```
static int cmd_q(char *args) {
    return -1;
}
```

添加一行代码后得到

```
static int cmd_q(char *args) {
    nemu_state.state = NEMU_QUIT;
    return -1;
}
```

- 最后, 查看现在 nemu 中输入 q 后的返回值:

```
wzhixing@ubuntu-22:~/ics2026/nemu$ make run
+ CC src/nemu-main.c
+ CC src/monitor/sdb/sdb.c
+ LD /home/wzhixing/ics2026/nemu/build/riscv32-nemu-interpret
/home/wzhixing/ics2026/nemu/build/riscv32-nemu-interpret --log=/home/wzhixing/
ics2026/nemu/build/nemu-log.txt
[src/utils/log.c:30 init_log] Log is written to /home/wzhixing/ics2026/nemu/buil
d/nemu-log.txt
[src/memory/paddr.c:50 init_mem] physical memory area [0x80000000, 0x87ffffff]
[src/monitor/monitor.c:51 load_img] No image is given. Use the default build-in
image.
[src/monitor/monitor.c:28 welcome] Trace: ON
[src/monitor/monitor.c:29 welcome] If trace is enabled, a log file will be gener
ated to record the trace. This may lead to a large log file. If it is not necess
ary, you can disable it in menuconfig
[src/monitor/monitor.c:32 welcome] Build time: 23:17:37, Sep  4 2026
Welcome to riscv32-NEMU!
For help, type "help"
(nemu) q
wzhixing@ubuntu-22:~/ics2026/nemu$ echo $?
0
```

图中结果表明我们已经完成了优美的退出.

- 对原先 nemu 运行程序完成后能够正常退出进行解释:  
原因是 nemu 运行完成后, 会将 `nemu_state` 设置为 `NEMU__END`, 进而符合 `is_exit_status_bad` 中的 `good` 条件, 从而能够顺利退出.

## VI. 实现单步执行, 打印寄存器, 扫描内存:

逐步添加 3 个函数: `cmd_si`、`cmd_info`、`cmd_x` 并实现对应功能.

- `cmd_si`: 这部分较容易, 只需让 CPU 执行 N 次即可;

```

static int cmd_si(char *args) {
    if (args == NULL) { // Default: N = 1
        cpu_exec(1);
    }
    else {
        uint64_t N;
        for (size_t i = 0; i < strlen(args); i++) {
            if (args[i] == ' ' || args[i] == '\0') break;
            if (!isdigit(args[i])) {
                printf("Syntax error near %c\n", args[i ? i - 1 : 0]);
                return 0;
            }
        }

        sscanf(args, "%lu", &N);

        cpu_exec(N);
    }
    return 0;
}

```

该命令运行示例:

```

(nemu) si 2
0x80000000: 00 00 02 97 auipc    t0, 0
0x80000004: 00 02 88 23 sb        zero, 0x10(t0)
(nemu) si
0x80000008: 01 02 c5 03 lbu        a0, 0x10(t0)

```

- **cmd\_info**: 这里需要分别处理打印寄存器和打印监视点, 根据第二个参数的取值来判断打印的内容.

- 1、对于打印寄存器而言, 直接调用 `isa_reg_display` 即可;
- 2、对于打印监视点而言, 我们需要单独编写一个相应的函数 `used_wp_display`, 后续步骤中我们再详细介绍该函数.

```

static int cmd_info(char *args) {
    char *arg = strtok(NULL, " ");

    if (strcmp(arg, "r") == 0) {
        isa_reg_display();
    } else if (strcmp(arg, "w") == 0) {
        used_wp_display();
    } else {
        printf("Undefined info command \"%s\"\n", arg);
    }
    return 0;
}

```

其中对于 riscv32 而言, `isa_reg_display` 的内容如下:

```

void isa_reg_display() {
    int i;
    for (i = 0; i < NR_REGS; i++) {
        printf("%-16s 0x%08x (%u)\n", regs[i], gpr(i), gpr(i));
    }
}

```

该命令 (info r) 运行示例:

```

(nemu) info r
$0          0x00000000 (0)
ra          0x00000000 (0)
sp          0x00000000 (0)
gp          0x00000000 (0)
tp          0x00000000 (0)
t0          0x80000000 (2147483648)
t1          0x00000000 (0)
t2          0x00000000 (0)
s0          0x00000000 (0)
s1          0x00000000 (0)
a0          0x00000000 (0)
a1          0x00000000 (0)
a2          0x00000000 (0)
a3          0x00000000 (0)
a4          0x00000000 (0)
a5          0x00000000 (0)
a6          0x00000000 (0)
a7          0x00000000 (0)
s2          0x00000000 (0)
s3          0x00000000 (0)

```

- `cmd_x`: 这里需要读两个参数, 分别是读取的 4 字节数 `N` 以及待求值的表达式 `EXPR`.

读取、处理参数并计算 EXPR 后, 就可以利用提供的 `paddr_read` 函数来读取内存中的数据.

```
static int cmd_x(char *args) {
    char *arg1 = strtok(NULL, " "); *arg2 = strtok(NULL, " ");
    if (arg1 == NULL || arg2 == NULL) {
        printf("Undefined x command\n");
        return 0;
    }
    else {
        int len;
        paddr_t addr;
        for (size_t i = 0; i < strlen(arg1); i++) {
            if (arg1[i] == ' ' || arg1[i] == '\0') break;
            if (!isdigit(arg1[i])) {
                printf("Syntax error near %c\n", arg1[i ? i - 1 : 0]);
                return 0;
            }
        }
        sscanf(arg1, "%d", &len);
        bool success = true;
        addr = expr(arg2, &success);

        if (success) {
            for (int i = 0; i < len; i++)
                printf("0x%x: 0x%08x\n", addr + 4 * i, paddr_read(addr + 4 * i, 4));
        }
        return 0;
    }
}
```

该命令运行示例:

```
(nemu) x 10 $t0
[src/monitor/sdb/expr.c:121 make_token] match rules[19] = "$t[0-6]" at position 0 with len 3: $t0
0x80000000: 0x00000297
0x80000004: 0x00028823
0x80000008: 0x0102c503
0x8000000c: 0x00100073
0x80000010: 0xdeadbe00
0x80000014: 0x57575757
0x80000018: 0x57575757
0x8000001c: 0x57575757
0x80000020: 0x57575757
0x80000024: 0x57575757
```

## VII. 实现算术表达式的词法分析:

- 添加规则:

```
} rules[] = {

    /* TODO: Add more rules.
     * Pay attention to the precedence level of different rules.
     */
    {"+", TK_NOTYPE}, // spaces
    {"\\+", '+'}, // plus
    {"\\-", '-'}, // minus
    {"\\*", '*'}, // multiply
    {"\\/", '/'}, // divide
    {"\\(", '('}, // left paren
    {"\\)", ')'}, // right paren
    {"==", TK_EQ}, // equal
    {"!=", TK_NEQ}, // not equal
    {"0[xX][0-9a-fA-F]+", TK_HEX}, // hexadecimal numbers
    {"[0-9]+", TK_NUM}, // decimal numbers
    {"&&", TK_AND}, // and
    {"\\$zero", TK_REG},
    {"\\$0", TK_REG},
    {"\\$x([0-9]|[1-2][0-9]|3[0-1])", TK_REG},
    {"\\$ra", TK_REG},
    {"\\$sp", TK_REG},
    {"\\$gp", TK_REG},
    {"\\$tp", TK_REG},
    {"\\$t[0-6]", TK_REG},
    {"\\$s[0-9]", TK_REG},
    {"\\$s1[0-1]", TK_REG},
    {"\\$a[0-7]", TK_REG}, // GPRs
    {"\\$pc", TK_REG}, // pc
    {"\\*", TK_DEREF}, // dereference
    {"\\-", TK_NEG} // negative
};
```

- 添加 token 信息:

```
while (e[position] != '\0') {
    /* Try all rules one by one. */
    for (i = 0; i < NR_REGEX; i++) {
        if (regexec(&re[i], e + position, 1, &pmatch, 0) == 0 && pmatch.rm_so == 0) {
            char *substr_start = e + position;
            int substr_len = pmatch.rm_eo;

            Log("match rules[%d] = \"%s\" at position %d with len %d: %.*s",
                i, rules[i].regex, position, substr_len, substr_len, substr_start);

            position += substr_len;

            /* TODO: Now a new token is recognized with rules[i]. Add codes
             * to record the token in the array `tokens'. For certain types
             * of tokens, some extra actions should be performed.
             */

            switch (rules[i].token_type) {
                case TK_NUM: case TK_HEX: case TK_REG:
                    strncpy(tokens[nr_token].str, substr_start, substr_len);
                    case '+': case '-': case '*': case '/':
                    case '(': case ')':
                    case TK_EQ: case TK_NEQ:
                    case TK_AND: case TK_DEREF:
                        tokens[nr_token].type = rules[i].token_type;
                        nr_token++;
                    case TK_NOTYPE:
                    default: break;
            }
            break;
        }
    }
}
```

## VIII. 实现算术表达式的递归求值:

按照框架代码实现 calc 函数:

```
static word_t calc(int p, int q, bool *success, bool* zero) {
    if (p > q) {
        *success = false;
        return 0;
    }
    else if (p == q) {
        Token cur_tok = tokens[p];
        word_t res = 0;
        switch (cur_tok.type) {
            case TK_NUM:
                for (int i = 0; i < strlen(cur_tok.str); i++) {
                    res = res * 10 + (cur_tok.str[i] - '0');
                }
                return res;
            case TK_HEX:
                for (int i = 0; i < strlen(cur_tok.str); i++) {
                    if (i == 0 || i == 1) continue; // ignore "0x"
                    if (cur_tok.str[i] >= '0' && cur_tok.str[i] <= '9')
                        res = res * 16 + (cur_tok.str[i] - '0');
                    else { // a-f & A-F
                        char ch = tolower(cur_tok.str[i]);
                        res = res * 16 + (ch - 'a' + 10);
                    }
                }
                return res;
        }
    }
}
```

```

else if (check_paren(p, q) == true) {
    return calc(p + 1, q - 1, success, zero);
}
else {
    int pos = main_op(p, q);
    if (pos == -1) {
        *success = false;
        return 0;
    }
    word_t val1 = 0, val2 = 0;
    if (binary_op(tokens[pos].type)) val1 = calc(p, pos - 1, success, zero);
    val2 = calc(pos + 1, q, success, zero);
    if (*success && !*zero) {
        switch (tokens[pos].type) {
            case '+': return val1 + val2;
            case '-': return val1 - val2;
            case '*': return val1 * val2;
            case '/':
                if (val2 == 0) {
                    *zero = 1;
                    return 0;
                }
                return val1 / val2;
        }
    }
}

```

其中 main\_op 用于寻找当前表达式主运算符, binary\_op 用于判断运算符是否为二元运算符, success 用于判断该表达式是否合法, zero 表示是否出现除以 0 的运算.

## IX. 实现表达式生成器:

在给出的 gen-expr.c 框架代码的基础上实现 gen\_num, gen\_op, gen\_rand\_expr 后, 运行 gen-expr 并把结果写入 input.

```

686 686
385 (385)
559 559
0 (-(-(-316)))/986
760 760
4294966567 -(729)
1709439299 (-(-194)/603*843)
296070916 -(761)+0x65 *(216*97)-((((546))))/0x11 /(0x9d )*33/368*((109)+-(-984))-((599+-(-316))/(580)))-((((413)+833+
917)+(332))/((995))*818-582*(854)/-(-(-281))/137+(766--(-314))*(588))-((-(-275)))/-(-455))*((-(-400))-((321-483-0x5
2 )))-(-647))+(-(-256))+(((331+0x5b /(183))/(0x5 ))) -574+-(-247)
23 (0x17 )
16647159 (((((-127)))/258))
915 915
183 0xb7
189140 (((772*0xf5 )))
473 391+0x52
83 (0x53 )
3089155914 0x75 +273*(-(-469))*(950-389)*(((0xa1 +(512+-716)))))+-(-726))/89-(((561)+0x3d ))+((((278/((0x10 ))))))*1
48)
124 0x7c
0 0x53 /(((0x2c ))-178*0x51 )
4294966328 -(968)
4292682952 ((((-(-69)))/0x9c --(-(-101))/((616)))/((157/0x7b ))-844)*(926/0x67 )+((0xe5 ))+462/914-(-(-544))/((0xc9 +0
x74 ))/((-(-954)))+956*0x73 +658+568-(471*((113/208+0x13 ))+0xc6 )*(-(-262)))+(((38)))/0xee +775)

```

某次运行完 gen-expr 后 input 中的内容

而后修改 nemu-main.c, 读入 input 的表达式, 并调用 expr 进行计算, 与正确结果比较.

```

int main(int argc, char *argv[]) {
| /* Initialize the monitor. */
#ifdef CONFIG_TARGET_AM
| am_init_monitor();
#else
| init_monitor(argc, argv);
#endif

    if (1) {
        init_regex();
        test_expr("tools/gen-expr/build/input");
        return 0;
    }

    /* Start engine. */
    engine_start();

    // printf("NEMU_STATE: %d\n", nemu_state.state);
    return is_exit_status_bad();
}

```

```

wzhixing@ubuntu-22:~/ics2026/nemu/tools/gen-expr/build$ ./gen-expr 100 > input
[src/monitor/sdb/expr.c:121 make_token] match rules[10] = "[0-9]+" at position 199 with len 3: 662
[src/monitor/sdb/expr.c:121 make_token] match rules[3] = "\*" at position 202 with len 1: *
[src/monitor/sdb/expr.c:121 make_token] match rules[9] = "0[xX][0-9a-fA-F]+" at position 203 with len 4: 0x59
[src/monitor/sdb/expr.c:121 make_token] match rules[0] = "+" at position 207 with len 1:
[src/monitor/sdb/expr.c:121 make_token] match rules[1] = "\+" at position 208 with len 1: +
[src/monitor/sdb/expr.c:121 make_token] match rules[10] = "[0-9]+" at position 209 with len 2: 93
[src/monitor/sdb/expr.c:121 make_token] match rules[6] = "\"\" at position 211 with len 1: )
[src/monitor/sdb/expr.c:121 make_token] match rules[1] = "\+" at position 212 with len 1: +
[src/monitor/sdb/expr.c:121 make_token] match rules[5] = "\"\" at position 213 with len 1: (
[src/monitor/sdb/expr.c:121 make_token] match rules[9] = "0[xX][0-9a-fA-F]+" at position 214 with len 4: 0xb4
[src/monitor/sdb/expr.c:121 make_token] match rules[0] = "+" at position 218 with len 1:
[src/monitor/sdb/expr.c:121 make_token] match rules[6] = "\"\" at position 219 with len 1: )
[src/monitor/sdb/expr.c:121 make_token] match rules[6] = "\"\" at position 220 with len 1: )
[src/monitor/sdb/expr.c:121 make_token] match rules[10] = "[0-9]+" at position 0 with len 1: 6
[src/monitor/sdb/expr.c:121 make_token] match rules[5] = "\"\" at position 0 with len 1: (
[src/monitor/sdb/expr.c:121 make_token] match rules[5] = "\"\" at position 1 with len 1: (
[src/monitor/sdb/expr.c:121 make_token] match rules[10] = "[0-9]+" at position 2 with len 3: 337
[src/monitor/sdb/expr.c:121 make_token] match rules[6] = "\"\" at position 5 with len 1: )
[src/monitor/sdb/expr.c:121 make_token] match rules[6] = "\"\" at position 6 with len 1: )
===== Pass: 100, Fail: 0 =====

```

某次测试运行结果

X. 扩展表达式求值的功能:

添加相应正则表达式识别规则并增加对应的运算/数据处理:

```

case TK_REG:
    bool flag = false;
    res = isa_reg_str2val(cur_tok.str, &flag);
    if (flag) return res;
    *success = false;
    return 0;

case TK_AND: return val1 && val2;
case TK_EQ: return val1 == val2;
case TK_NEQ: return val1 != val2;
case TK_NEG: return (word_t)(-val2);
case TK_DEREF: return paddr_read(val2, 4);

```

## XI. 实现监视点池的管理:

```

WP* new_wp() {
    if (!free_) printf("No free watchpoints\n"), assert(0);

    WP *wp = free_;
    free_ = free_>next;
    wp->next = head, head = wp;

    return wp;
}

void free_wp(int wp_NO) {
    if (head == NULL) return;

    WP *cur = head, *prev = NULL;
    while (cur && cur->NO != wp_NO) {
        prev = cur, cur = cur->next;
    }

    if (cur == NULL) return;

    if (prev) prev->next = cur->next;
    else head = cur->next;

    cur->next = free_, free_ = cur;

    memset(cur->expr, 0, sizeof(cur->expr)), cur->val = 0;
}

```

## XII. 实现监视点:

首先编写 `cmd_w`, 在此函数中通过调用 `new_wp` 来创建一个监视点.

其次, 为了实现监视点的全部功能, 我们还需要以下 2 个函数: 一个是 `check_all_wp`, 另一个是前文提及过的 `used_wp_display` 函数.

第一个函数是在 `trace_and_difftest` 函数中发挥作用的, 它的功能主要是判断是否有监测点的表达式的值相较于 CPU 执行当前这条指令之前发生了变化;

```
// Check all the watchpoints
bool check_all_wp() {
    if (head == NULL) return 0;

    WP *cur = head;
    bool change = false;

    while (cur != NULL) {
        bool success = true;
        word_t new_val;
        if (cur->val != (new_val = expr(cur->expr, &success)) ) {
            change = 1;
            printf("Watchpoint %d: %s\n", cur->NO, cur->expr);
            printf("Old value: 0x%x\n", cur->val);
            printf("New value: 0x%x\n", new_val);
            cur->val = new_val;
        }
        cur = cur->next;
    }

    return change;
}
```

第二个函数是在 `cmd_info` 中发挥作用的, 如前文所述. 它的功能是打印所有监视点的表达式的内容.

```
void used_wp_display() {
    if (head == NULL) {
        printf("No watchpoints\n");
        return;
    }
    printf("%-10sWhat\n", "Num");
    WP *cur = head;

    while (cur != NULL) {
        printf("%-10d%s\n", cur->NO, cur->expr);
        cur = cur->next;
    }
}
```

最后再编写函数 `cmd_d`, 在此函数中通过调用 `free_wp` 来删除一个监视点.

监视点功能示例:

```
(nemu) w $t0
[src/monitor/sdb/expr.c:121 make_token] match rules[19] = "\\$t[0-6]" at position 0 with len 3: $t0
Watchpoint 0: $t0
(nemu) w $a0
[src/monitor/sdb/expr.c:121 make_token] match rules[22] = "\\$a[0-7]" at position 0 with len 3: $a0
Watchpoint 1: $a0
(nemu) w $pc == 0x8000000c
[src/monitor/sdb/expr.c:121 make_token] match rules[23] = "\\$pc" at position 0 with len 3: $pc
[src/monitor/sdb/expr.c:121 make_token] match rules[0] = "+" at position 3 with len 1:
[src/monitor/sdb/expr.c:121 make_token] match rules[7] = "==" at position 4 with len 2: ==
[src/monitor/sdb/expr.c:121 make_token] match rules[0] = "+" at position 6 with len 1:
[src/monitor/sdb/expr.c:121 make_token] match rules[9] = "0[xx][0-9a-fA-F]+" at position 7 with len 10: 0x8000000c
Watchpoint 2: $pc == 0x8000000c
(nemu) info w
Num      What
2        $pc == 0x8000000c
1        $a0
0        $t0

0x8000000c: 00 10 00 73 ebreak
[src/monitor/sdb/expr.c:121 make_token] match rules[23] = "\\$pc" at position 0 with len 3: $pc
[src/monitor/sdb/expr.c:121 make_token] match rules[0] = "+" at position 3 with len 1:
[src/monitor/sdb/expr.c:121 make_token] match rules[7] = "==" at position 4 with len 2: ==
[src/monitor/sdb/expr.c:121 make_token] match rules[0] = "+" at position 6 with len 1:
[src/monitor/sdb/expr.c:121 make_token] match rules[9] = "0[xx][0-9a-fA-F]+" at position 7 with len 10: 0x8000000c
Watchpoint 2: $pc == 0x8000000c
Old value: 0x1
New value: 0x0
[src/monitor/sdb/expr.c:121 make_token] match rules[22] = "\\$a[0-7]" at position 0 with len 3: $a0
[src/monitor/sdb/expr.c:121 make_token] match rules[19] = "\\$t[0-6]" at position 0 with len 3: $t0
You hit a watchpoint.
(nemu) d 1
(nemu) info w
Num      What
2        $pc == 0x8000000c
0        $t0
```

XIII. 尝试通过目录定位关注的问题:

根据目录我们可以定位 `selector` 在手册中的位置 (5.1.3 节):

|                                          |           |
|------------------------------------------|-----------|
| <b>CHAPTER 5 MEMORY MANAGEMENT .....</b> | <b>91</b> |
| 5.1 SEGMENT TRANSLATION .....            | 92        |
| 5.1.1 Descriptors.....                   | 92        |
| 5.1.2 Descriptor Tables.....             | 94        |
| 5.1.3 <u>Selectors</u> .....             | 96        |
| 5.1.4 Segment Registers.....             | 97        |
| .....                                    | ..        |

再了解 `selectors` 小节中的内容即可:

### 5.1.3 Selectors

The selector portion of a logical address identifies a descriptor by specifying a descriptor table and indexing a descriptor within that table. Selectors may be visible to applications programs as a field within a pointer variable, but the values of selectors are usually assigned (fixed up) by linkers or linking loaders. [Figure 5-6](#) shows the format of a selector.

**Index:** Selects one of 8192 descriptors in a descriptor table. The processor simply multiplies this index value by 8 (the length of a descriptor), and adds the result to the base address of the descriptor table in order to access the appropriate segment descriptor in the table.

**Table Indicator:** Specifies to which descriptor table the selector refers. A zero indicates the GDT; a one indicates the current LDT.

**Requested Privilege Level:** Used by the protection mechanism. (Refer to [Chapter 6](#))

Because the first entry of the GDT is not used by the processor, a selector that has an index of zero and a table indicator of zero (i.e., a selector that points to the first entry of the GDT), can be used as a null selector. The processor does not cause an exception when a segment register (other than CS or SS) is loaded with a null selector. It will, however, cause an exception when the segment register is used to access memory. This feature is useful for initializing unused segment registers so as to trap accidental references.

#### XIV. 实验报告中需要回答的必答题:

- 程序是个状态机: 这个问题在本报告中已经给出相应回答;
- 理解基础设施:

根据题目中的信息推算, 这学期在调试上花费的时间是:

$$500 \times 90\% \times 30 \times 20 = 270000s = 75h$$

利用简易调试器可以节省的时间是:

$$500 \times 90\% \times (30 - 10) \times 20 = 180000s = 50h$$

- RTFM1:

我选择的 ISA 是 riscv32, 对应问题在手册中的位置是:

– riscv32 的指令格式:

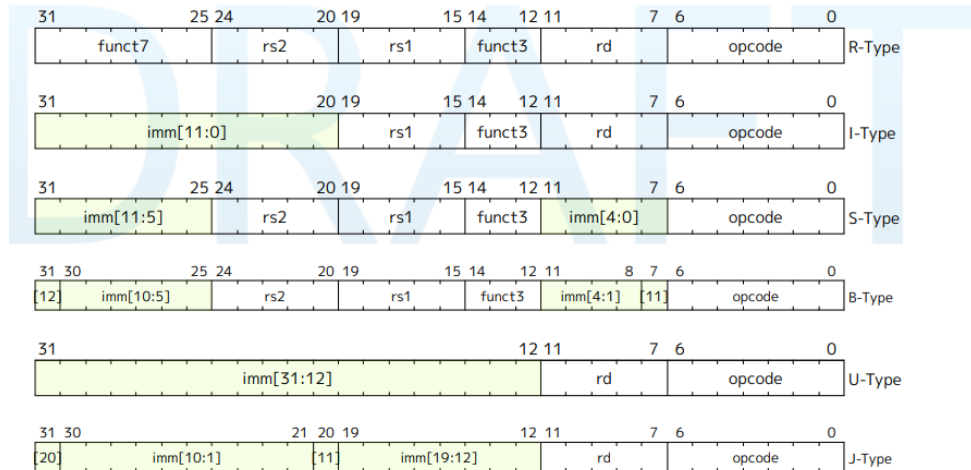
在 Unprivileged Architecture 卷中:

#### 2.2. Base Instruction Formats

In the base RV32I ISA, there are four core instruction formats (R/I/S/U), as shown in [Base instruction formats](#). All are a fixed 32 bits in length. The base ISA has **IALIGN=32**, meaning that instructions must be aligned on a four-byte boundary in memory. An instruction-address-misaligned exception is generated on a taken branch or unconditional jump if the target address is not **IALIGN-bit** aligned. This exception is reported on the branch or jump instruction, not on the target instruction. No instruction-address-misaligned exception is generated for a conditional branch that is not taken.

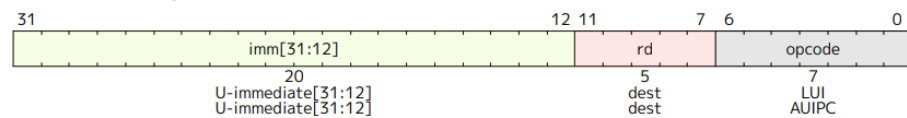
### 2.3. Immediate Encoding Variants

There are a further two variants of the instruction formats (B/J) based on the handling of immediates, as shown in [Base instruction formats immediate variants](#).



– LUI 指令的行为:

在 Unprivileged Architecture 卷中:



The RISC-V Instruction Set Manual Volume I | © RISC-V

2.4. Integer Computational Instructions | Page 27

**LUI** (load upper immediate) is used to build 32-bit constants and uses the U-type format. LUI places the 32-bit U-immediate value into the destination register *rd*, filling in the lowest 12 bits with zeros.

– mstatus 寄存器的结构:

在 Privileged Architecture 卷中:

#### 3.1.6. Machine Status Registers (**mstatus** and **mstatush**)

The **mstatus** register is an MXLEN-bit read/write register formatted as shown in [Figure 7](#) for RV32 and [Figure 8](#) for RV64. The **mstatus** register keeps track of and controls the hart's current operating state. A restricted view of **mstatus** appears as the **sstatus** register in the S-level ISA.

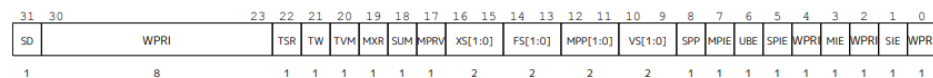


Figure 7. Machine-mode status register (**mstatus**) for RV32

• shell 命令:

– 统计.c 和.h 文件代码行数:

主要依赖 find 命令 (找特定文件) 和 wc 命令 (计数) 来完成:

```
wzhtxng@ubuntu-22:~/lcs2026/newu$ find . \( -name "*.c" -o -name "*.h" \) -print0 | xargs -0 wc -l | tail -1
267311 总计
```

- 和框架代码相比, 在 PA1 中编写代码的数量:

使用 git diff 来完成:

```
wzhixing@ubuntu-22:~/ics2026/newu$ git diff pa0 --numstat | awk '{a+=$1; d+=$2} END {print "added:", a, "deleted:", d}'  
added: 496 deleted: 23
```

- 统计.c 和.h 文件非空行行数:

额外添加一个 “grep -v ‘^\$’ ” 过滤空行即可:

```
wzhixing@ubuntu-22:~/ics2026/newu$ find . \( -name "*.c" -o -name "*.h" \) | xargs cat | grep -v '^$' | wc -l  
230953
```

- RTFM2:

- **-Wall**: 作用是开启绝大部分常见的编译器警告;

```
-Wall  
This enables all the warnings about constructions that some users  
consider questionable, and that are easy to avoid (or modify to  
prevent the warning), even in conjunction with macros. This also  
enables some language-specific warnings described in C++ Dialect  
Options and Objective-C and Objective-C++ Dialect Options.
```

- **-Werror**: 作用是把所有的警告转变为错误.

```
-Werror  
Make all warnings into errors.
```

### 3 实验心得和想法

PA1 是整个 PA 的最初阶段, 本阶段核心内容就是在做很朴素的一些事: 阅读框架代码、编写简易调试器、阅读手册和学习运用 bash 命令等, 但是这些内容都是至关重要的, 为最终实现一个完整的计算机系统打下基础, 同时能初步感受到阅读、编写一个小项目代码的难度和压力.

完成整个 PA1 还是需要花费一定的时间的, 虽然在此阶段调试 bug 并不困难. 在 sdb 的设计过程中, 我也逐步学会更熟练地运用 gdb 调试, 有所收获.